

Parallel Join Algorithms

1. Introduction

Classical Join Algorithms

Database joins combine two tables based on a shared key. Traditionally, three main join algorithms are used:

1. **Index Nested Loop Join** – Uses an index on one input table for fast lookups.
2. **Sort-Merge Join** – Sorts both tables by the join key, then merges them like in merge sort.
3. **Hash Join** – Uses a hash table to quickly match join keys between two tables.

In the context of parallel database systems, the traditional join algorithms are updated to make better use of multi-core processors and large main memory. The goal of parallelization is to divide the work among several threads and cores, while hardware-aware methods help reduce problems with cache and memory performance.

2. History and New Hardware

- Most architectural ideas in today's CPUs, such as multiple processors (cores), pipelines, and various forms of parallelism, were first explored in historical supercomputers.
 - 1960, Atlas, Manchester University - Cited as the first computer to use a sophisticated form of paged virtual memory
 - 1964, CDC 6600, Seymour Cray - Considered the first successful supercomputer
 - 1976, Cray-1, 1985, Cray-2 - Featured multiple processors and extensive use of pipelines, vector processing capabilities of the Cray systems are direct predecessors to modern SIMD extensions
 - 1980, Connection Machine, MIT - Highly parallel computer, featuring 65,536 processors
- Many of the core concepts used in new, highly parallel DBMSs were already present in the 1980s.
 - 1981, Grace (Tokyo University) - Associated with early ideas for parallel hash join algorithms. The concept of the Grace Hash Join is a direct ancestor of the modern partitioned and radix hash joins.
 - 1985, Gamma Database Machine (Wisconsin University) - A parallel database system using a shared-nothing architecture that implemented parallel relational operators and advanced parallel join methods like the partitioned hash join.
 - 1988, Bubba (University of Texas) - Advanced the shared-nothing architecture and contributed to techniques for handling data skew in parallel systems.

Modern systems are **main-memory databases**, most data fits in RAM.

These developments directly led to modern main-memory join algorithms such as the No Partitioning Join (NPJ) and the Radix Hash Join (RHJ). These algorithms are made to work well on multi-core processors while also paying attention to how the cache and the TLB affect performance.

According to Balkesen et al. (2013), even with modern CPUs, the way we tune the algorithm to the hardware — for example, the size of the cache, TLB, and memory bandwidth — has a strong effect on how fast the joins run. Their study showed that hardware-aware algorithms perform better than those that ignore hardware details.

Hardware Features

2.1. Multi-Core CPUs

- Modern processors have **multiple cores**, allowing **true parallel execution** of tasks.
- Each core can execute its own thread, so database operations (e.g., joins, scans) can be parallelized across cores.
- Each core may support Simultaneous Multi-Threading (SMT), allowing two or more threads to share the same physical core (e.g., Intel's Hyper-Threading). Efficient mapping of threads to cores is important for reducing contention and achieving linear speedup in parallel join algorithms.

2.2. Cache Hierarchy

Cache Level	Location	Speed (vs RAM)	Typical Use
L1 Cache	Per core	~100× faster	Holds the most frequently used data/instructions.
L2 Cache	Per core	~25× faster	Stores recently used data; slightly larger but slower than L1.
L3 Cache	Per socket (shared by cores)	~2–3× faster	Shared among cores for inter-core communication.

Key point: To manipulate data quickly, keep it as close to the CPU as possible (ideally in L1/L2 cache).

Memory access, not CPU speed, is now often the **main bottleneck** in performance.

Cache-aware join algorithms, like the Radix Hash Join, are designed so that each partition can fit inside the CPU cache. This helps to avoid many random memory accesses, which slow down the process. These algorithms also make use of CPU prefetching and sequential memory access, which help the processor read data faster and keep the overall performance high.

2.3. Vector Processing (SIMD – Single Instruction, Multiple Data)

- SIMD (e.g., **Intel AVX-512**) allows a single CPU instruction to process **multiple data elements in parallel**.
- Supports **predication**, **arithmetic**, **shuffle**, and **memory** operations on vectors of data.
- Greatly speeds up operations like comparisons, aggregations, and joins.

Drawback: While SIMD can improve performance, using wide vector instructions (like AVX-512) may cause **CPU frequency downscaling** (“thermal throttling”)

2.4. Main Memory and Clusters

- Modern database systems often have **large amounts of main memory**, especially in **clustered environments**.
- This enables **in-memory DBMSs**, where most data resides in RAM instead of disk, further reducing latency.
- However, as memory hierarchies become deeper and systems transition to Non-Uniform Memory Access (NUMA), data placement becomes critical. NUMA-aware join implementations ensure that each thread primarily accesses memory local to its CPU socket, avoiding cross-socket bandwidth limitations.

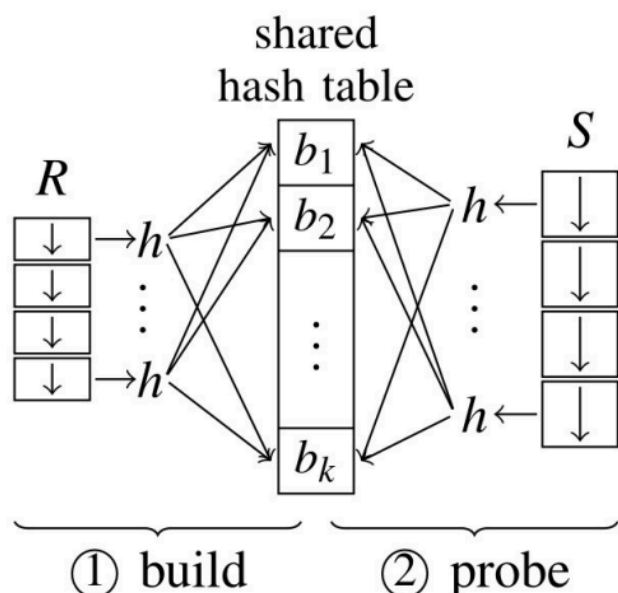
3. Parallel Hash Joins

3.1. No Partitioning Hash Join (NPJ)

- NPJ is the **simplest parallel version of the hash join** algorithm
- It's a **direct parallelization** of the classical hash join, it doesn't rely on hardware-specific parameters like cache or NUMA topology.
- Each thread processes a chunk of data independently, but all threads share the same hash table.

Algorithm Steps

1. **Build Phase:**
 - All threads insert tuples from relation **R** into a **shared hash table** in memory.
 - This phase builds the hash structure based on join keys.
2. **Probe Phase:**
 - Threads scan relation **S** and **probe** the hash table to find matching tuples in **R**.



- Each thread works on its own portion of **S**, but all threads access the same hash table.

Synchronization Issue

- Since all threads insert into **one shared hash table**, they must **synchronize access** to prevent race conditions.
- Synchronization is achieved using:
 - **Locks or latches** (lightweight semaphores for in-memory data structures).
 - **Atomic operations** such as **Compare-And-Swap (CAS)** to safely update shared memory.
- However, more threads cause **contention and polling**, which can reduce performance.
- In practice, each bucket in the shared hash table is protected by a latch. When a thread wants to insert a tuple, it must first acquire the latch for that bucket. This reduces race conditions but introduces waiting time when several threads target the same bucket. The probe phase is read-only, so no synchronization is needed there.

Cache Effects

- If the hash table **fits into the CPU cache** (especially **L2 cache**), performance is excellent.
- If it's **too large for cache**, each access triggers a **cache miss**, forcing slow main memory access.
- **Optimization:** Use **smaller hash tables** or **cache-sized chunks** to keep data in cache and reduce memory latency.
 - (As shown in research: Shatdal et al. 1994; Boncz et al. 1999)
- Modern CPUs attempt to hide some cache latency through out-of-order execution and prefetching, but these mechanisms are not sufficient for random hash table accesses. Therefore, cache-friendly data organization remains crucial.

Translation Lookaside Buffer (TLB)

- The **TLB** is a small hardware cache that stores **virtual-to-physical memory mappings**.
- Each memory **page** (usually 4 KB, sometimes up to 16 KB) has an entry in the TLB.
- When the number of memory pages accessed **exceeds the number of TLB entries**, **TLB misses** occur, slowing memory access.

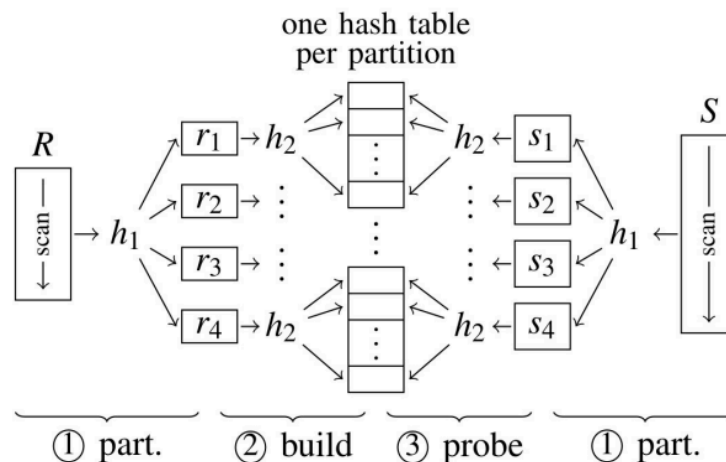
Implications for Joins:

- Each partition or chunk of data may reside on a different memory page → too many partitions → too many TLB lookups.
- To reduce TLB misses:
 - Limit the number of partitions (ideally $\leq \# \text{TLB entries}$).
 - Use **multi-pass or radix partitioning**, which became standard in **radix join** algorithms.

- Modern operating systems sometimes support large memory pages (e.g., 2 MB “huge pages”), which can reduce the number of TLB misses by mapping more data with fewer entries. Using large pages can noticeably improve hash join performance.

3.2 Partitioned Hash Join (Grace Join)

- The **Partitioned Hash Join** improves over the No Partitioning Hash Join by reducing **cache misses**, **TLB misses**, and **synchronization overhead**.
- Originates from the **Grace Hash Join** algorithm (University of Tokyo, 1985).
- Commonly used in both **disk-based** and **main-memory** database systems.



1. Partition Phase

- Both relations R and S are divided into partitions using a **hash function (h_1)** on the join key.
 - Tuples from $R \rightarrow$ partitions $r_1, r_2, \dots, r_n$
 - Tuples from $S \rightarrow$ partitions $s_1, s_2, \dots, s_n$
- Only partitions with the **same hash value** can contain matching tuples:
- Each partition is small enough to **fit into cache**, reducing cache and TLB misses.
- Each partition r_i** is assigned to a **separate thread**, which builds and probes its own hash table.

Benefits:

- Each thread works on separate data $\rightarrow$ **no synchronization needed**.
- Each hash table fits in cache $\rightarrow$ **fewer cache misses**.
- Works efficiently on multi-core CPUs.

Challenge:

- If the number of partitions is too large, it may exceed the **TLB capacity**, causing **TLB misses** and **page faults**.

- This can be mitigated by **multi-pass partitioning** or using **histograms** to balance partition sizes.

Grace Hash Join was originally designed for disk-based systems, where partitions were written to disk before probing. In main-memory systems, the same logic is used to optimize for cache instead of disk I/O.

Also, partition size is often chosen based on cache capacity (e.g., L2 cache) to ensure that hash tables stay cache-resident during probing.

2. Build Phase

- For each partition r_i :
 - A **local hash table** is created (using a second hash function h_2).
 - Each thread inserts its tuples independently → **no locking required**.
- The hash table for each partition fits entirely into **CPU cache**.
- This step allows perfect parallelism because threads work independently and never share memory. Each thread can use optimized memory allocation to keep its hash table aligned in cache lines, avoiding false sharing.

3. Probe Phase

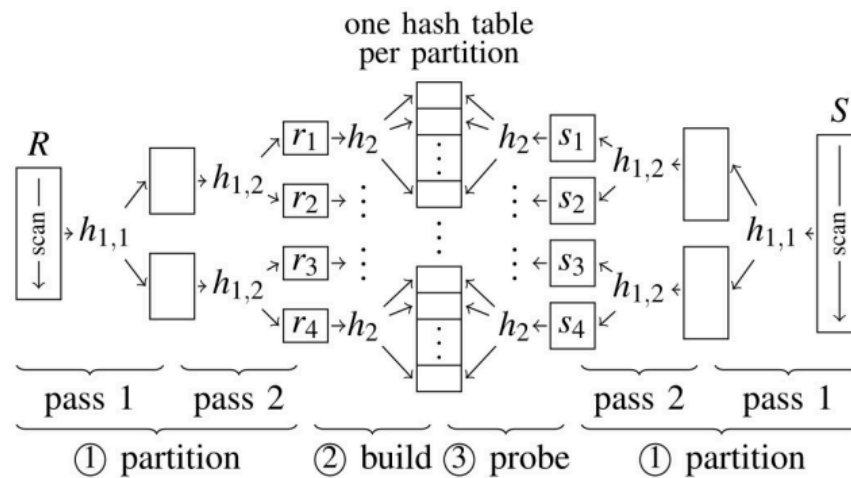
- Each thread scans its corresponding s_i partition and probes the **hash table** of r_i using the same hash function h_2 .
- Matches are found and output in parallel.
- Since each thread works on isolated data, **contention is eliminated**.
- The output phase can also be parallelized using thread-local buffers to avoid contention when writing the result tuples. These buffers are later combined into the final output.

Disk-Based vs Main-Memory Use

- **Originally (Grace Hash Join)** was designed for **disk-based systems** with limited memory, partitions were written to disk, then reloaded.
- In **modern in-memory systems**, partitioning is done to **fit partitions into CPU caches**, not disks.

3.3 Radix Hash Join

- Radix Hash Join is a cache-optimized improvement of the Partitioned Hash Join.
- It minimizes cache and TLB misses by partitioning data in multiple passes, each refining the previous one.
- Instead of splitting the data once, RHJ performs several partitioning passes.
- Typically, two or three passes are enough to create partitions that fit comfortably in cache and stay within TLB limits.



1. Partition Phase

- Both relations R and S are divided using a hash function.
- The partitioning continues in multiple passes until each partition is small enough to fit into the CPU's L2 cache.
- This avoids excessive cache and TLB misses.
- The fan-out per pass is limited by the number of TLB entries, ensuring efficient memory mapping.
- Before writing data to partitions, histograms are built to estimate partition sizes.
- This allows threads to precompute output memory locations and avoid synchronization.
- Each thread writes to its own region, so no locking or atomic operations are required.
- Radix partitioning uses the lower bits of the hash function in each pass ($h_1, h_2, \dots$).
- Each pass increases the number of partitions exponentially until they are small enough for the cache. This multi-pass refinement prevents TLB overflow and keeps memory access patterns sequential.

2. Build Phase

- After partitioning, each partition of R is processed by one thread, which builds a local hash table.
- Since each thread operates on its own data, synchronization is unnecessary, and the hash tables fit in cache.
- To improve performance, memory for hash tables is preallocated using histogram-based size estimation. This avoids expensive dynamic allocations during the build phase.

3. Probe Phase

- Each partition of S is scanned and probed against its matching R partition.
- Because partitions are small, cache-resident, and isolated, this phase runs very efficiently with almost no contention between threads.

The **time complexity** of Radix Hash Join is

$$O((|R| + |S|) \cdot \log |R|).$$

The number of passes depends on the logarithm of the relation size, with the base

corresponding to the number of TLB entries. In practice, R is usually smaller, so optimization focuses on keeping R 's partitions cache-sized.

Hardware Awareness

RHJ is deeply tuned to hardware behavior. Each partition is small enough to fit in cache, and the number of active partitions never exceeds the number of TLB entries. Because each thread works on separate memory regions, synchronization and cache line contention are avoided. Memory access patterns are sequential and cache-friendly, further improving speed.

3.4 Parallel Radix Hash Join (PRHJ)

- PRHJ is the **parallel version of Radix Hash Join**, designed to fully exploit multi-core CPUs while keeping data cache-friendly.
- It extends RHJ by mapping partitions and tasks efficiently to threads, minimizing contention and memory stalls.

1. Task Mapping and Partitioning

- Relations **R** and **S** are divided into **sub-relations** assigned to individual threads.
- **First Pass:**
 - All threads jointly partition the data into a **shared set of partitions**.
 - Because multiple threads access the same partitions, there is a **risk of contention**.
 - The number of partitions in this first pass is limited by **hardware parameters** (e.g., TLB entries, cache size).
- **Later Passes:**
 - Each thread works on its **own set of partitions**.
 - Threads write to dedicated memory ranges within each partition, eliminating contention.
 - Thread-local statistics (like how many tuples hash to each partition) are used to precompute memory locations efficiently.
- This histogram-based approach allows the system to precompute “prefix sums” for each thread, defining where each thread writes its output without overlap. This method removes the need for locks during partitioning.

2. Build Phase

- Each thread builds **hash tables** for its assigned partitions.
- Partitions are small enough to **fit in CPU cache**, so memory access is fast.
- **No synchronization** is needed because each thread operates on its own partition and hash table.
- During this phase, dynamic task scheduling (e.g., task queues) can be used to balance workload among threads, especially if partition sizes differ.

3. Probe Phase

- Each thread scans its corresponding **S partition** and probes the hash table of the matching **R partition**.
- Since each thread works on a unique pair of partitions, **no contention occurs**, even when writing results to the output table.
- NUMA-aware scheduling can further improve performance by ensuring that each thread processes partitions located in its local memory region, minimizing remote memory access delays.

Key Advantages

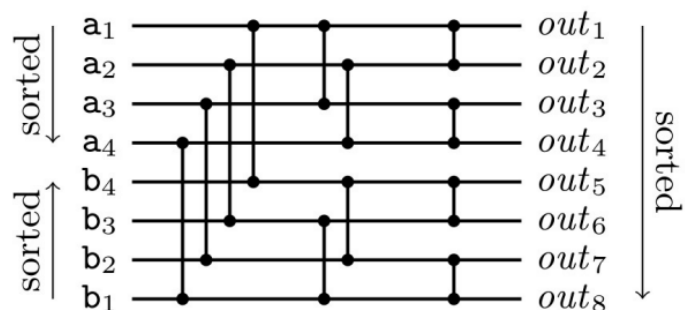
- Efficiently uses **multi-core parallelism** while respecting **cache and TLB limits**.
- Thread-local memory allocation and precomputed output ranges eliminate contention.
- Both build and probe phases are **fully independent**, allowing **highly parallel execution**.
- Extremely cache-optimized, suitable for **main-memory databases**.

4. Parallelizing Sort with SIMD

Sorting is an important part of sort-merge joins, where both relations must be sorted before merging. Sorting gets much faster when we use SIMD and split the work across many threads. The goal is to keep data in cache as long as possible and avoid slow main-memory access.

1. Sorting Networks

- A **sorting network** is a fixed sequence of comparators that perform **min/max operations**. Because the sequence is fixed, the CPU does not waste time on branches.
- No branching is required, making them ideal for **SIMD execution**.
- Example: Sorting 4 numbers in SIMD registers (each register holds 4 elements) can give a **2–3× speedup**.
- Implementation steps:
 - Comparators traverse the data from left to right.
 - Each comparator outputs the smaller value on top, larger at the bottom.
 - Can be implemented entirely with min/max operations — very fast.
- For SIMD: multiple vectors are loaded and then **transposed** (shuffle operations) before sorting.
- Sorting 4 vectors with SIMD can achieve **~2.7× speedup** over scalar sorting despite shuffle overhead.



In practice, 2.5–3× is common; more width (e.g., AVX-512) helps only if memory and cache can keep up.

2. Merging Sorted Runs

- **Bitonic Merge Networks** merge two pre-sorted lists using SIMD **min/max and shuffle operations**. Bitonic merge is efficient because it repeats a small pattern of SIMD steps. This keeps the pipeline busy.
- These networks can serve as **kernels** for larger merging algorithms. Larger merges call the bitonic kernel many times, emitting the smaller vector each round.
- For larger lists, small merge networks are applied hierarchically. This hierarchical approach keeps most work inside L1/L2 cache before touching main memory.
- This allows efficient merging using vectors instead of scalars, improving SIMD utilization. Using vectors reduces comparisons per element and lowers control-flow cost.

3. Sorting and Memory Hierarchy

- Modern hardware has **multi-level cache hierarchies**, so sorting must be split into phases:
 1. **In-register sorting**: Small runs that fit into SIMD registers are sorted first using sorting networks.
 2. **In-cache sorting**: Small runs are merged while still fitting in L1/L2 cache.
 3. **Out-of-cache sorting**: Final merges that access main memory, which is the slowest phase.
- Goal: keep as much work as possible in **cache** to reduce RAM access and improve speed.

4. Balancing Computation and Memory Bandwidth

- Sorting speed is often limited by **memory bandwidth**, not CPU power.
- Example: Merging 8 tuples in a bitonic merge requires ~29 CPU cycles, corresponding to ~10 GB/s per thread
- Multiple threads can quickly saturate memory bandwidth.
- **Optimization**: Multi-way merging reduces memory round-trips by merging several runs at once instead of only two.
- Implementation: Multiple two-way merge units connected via **FIFO queues**, sized to fit entirely in cache, maximize bandwidth efficiency.

5. SIMD-Optimized Sorting Algorithms

- **AVX sorting algorithms** can achieve 2.5–3× speedup over standard scalar C++ sort.
- Use **vector registers** for in-register sorting and **bitonic merge networks** for hierarchical merging.
- Efficient use of cache and multi-threading ensures the CPU does not stall waiting for memory.

5. Sort-Merge Join and NUMA

Sort-merge join works in two main steps: sorting both relations on the join key, and then merging them to find matching tuples. The algorithm is fully deterministic and performs best when both inputs are already sorted or can be sorted quickly using SIMD and parallel sorting methods.

- **NUMA (Non-Uniform Memory Access):** Multi-socket systems where each socket has its **own local memory**.
- **Local memory:** fast, low latency; **Remote memory:** slower, limited bandwidth.
- NUMA-aware algorithms are critical for modern multi-socket systems.
- **Local memory** access should be maximized to reduce latency and avoid bandwidth bottlenecks, especially in memory-intensive operations like Sort-Merge Join.

6. Sort-Merge Joins (SMJ)

6.1 Classic Sort-Merge Join (SM Join)

1. **Sort Phase:**
 - Sort both relations **R** and **S** by the join key.
 - Sorting is the **dominant cost** in the join.
 - External merge sort can be used for large datasets.
2. **Merge Phase:**
 - Merge the sorted relations to find matching tuples.
3. **Modern Optimizations:**
 - Use **SIMD instructions** to accelerate both sorting and merging.
 - SIMD registers (e.g., 4×4 or 8×8 elements) allow parallel processing of multiple values in a single instruction.

Key point: Sort-Merge join performance depends mostly on sorting efficiency, and modern CPUs exploit SIMD to speed it up.

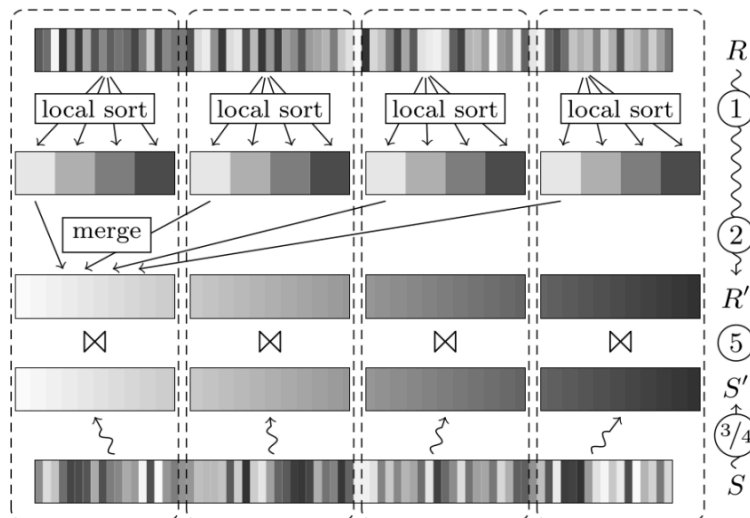
6.2 Multi-way Sort-Merge Join

- Combines **data parallelism** (SIMD) with **thread parallelism** across **NUMA regions**.
- **Step 1:** Distribute R and S across NUMA nodes; each thread sorts its **local chunk** using AVX/SIMD instructions.
- **Step 2:** Shuffle and merge data across NUMA regions to produce a **globally sorted R'**.
- **Step 3:** Repeat the same process for S to get **S'**.
- **Step 4:** Each thread performs a **single-pass merge join** on its NUMA-local R' and S'.

Key Features:

- After initial partitioning, threads work independently → **no synchronization** required.

- Partitioning fan-out typically matches the number of threads (e.g., 64–128).
- Multi-way merging overlaps data transfer and computation to balance NUMA interconnect usage.
- Fully **NUMA- and SIMD-optimized**, enabling high throughput for in-memory joins.



6.3 Multi-Pass Sort-Merge Join

- Similar to Multi-way Sort Merge Join, but replaces the global multi-way merge with **successive two-way merges**.
- **Phase 1:** Initial sorting of NUMA-local chunks using SIMD.
- **Phase 2:**
 - Two-way merges occur **during data transfer** from remote to local memory.
 - Reduces the number of runs by half in each iteration.
- **Phase 3:** Remaining merges are done **entirely in local memory**.
- Designed for systems where **memory bandwidth is limited**, reducing pressure on interconnects.

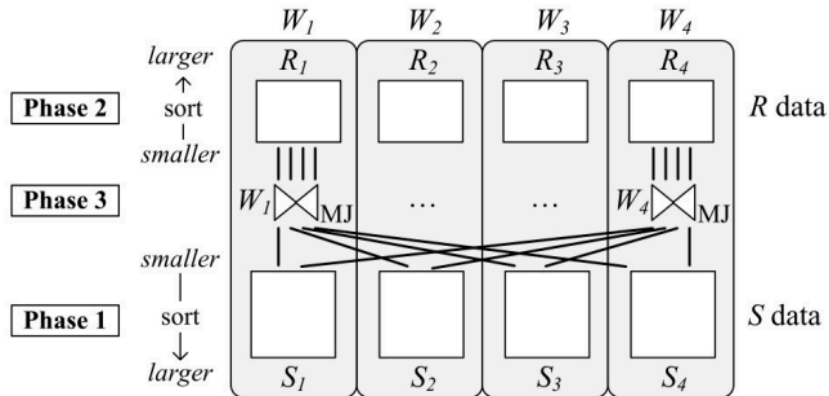
6.4 Massively Parallel Sort-Merge Join (MPSM)

Two versions:

1. Basic MPSM (B-MPSM)

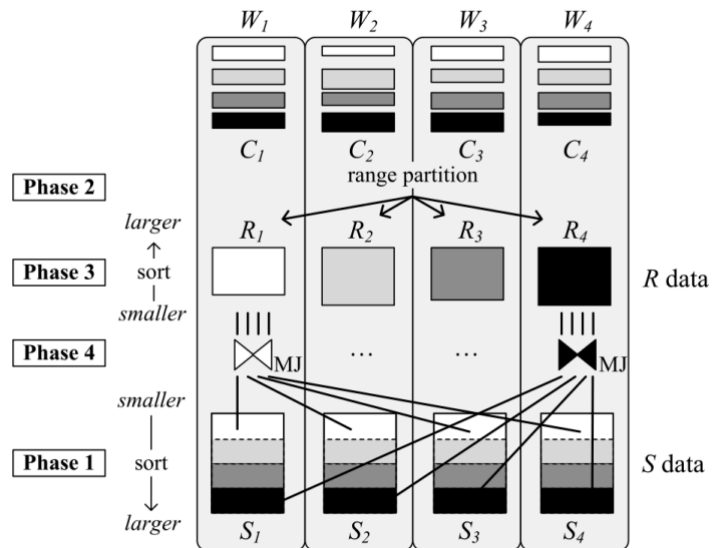
- Each thread sorts its chunk of R and S in parallel.
- Threads then join their R chunk with all S chunks sequentially.
- **Features:**
 1. Excellent scalability and minimal synchronization.

2. Sequential scans are fast due to **cache locality and prefetching**.
 3. Skew-resistant: all threads process equal-sized chunks.
- **Limitations:**
 1. AVX/SIMD sorting is less effective due to large join keys and payloads.
 - **Phases:**
 1. Distribute chunks of R and S to threads.
 2. Sort each chunk locally.
 3. Each thread joins its R chunk with all S chunks sequentially.



2. Range-Partitioned MPSM (P-MPSM)

- Adds a **range-partitioning step** before sorting to improve load balancing.
- **Algorithm:**
 - **S is chunked and sorted** locally by threads.
 - **R is range-partitioned** using histograms to ensure balanced partitions even with skewed data.
 - Threads sort their **local R partitions**.
 - Each thread joins its R partition with all S runs sequentially.
- **Advantages:**
 - Efficiently handles skewed data.
 - Preserves NUMA locality and minimizes synchronization.



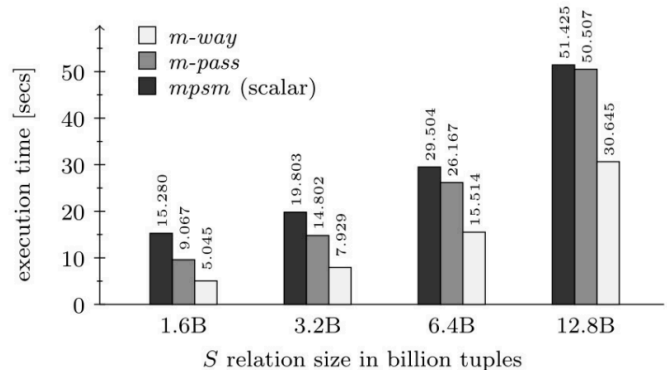
7. Experimental Findings

Sort-Merge Join (SMJ) Algorithm Comparison

This section compares the performance of three Sort-Merge Join variants: **m-way** (Multi-way SMJ), **m-pass** (Multi-pass SMJ), and **mpsm** (Massively Parallel SMJ).

Execution Time (Time vs. Size)

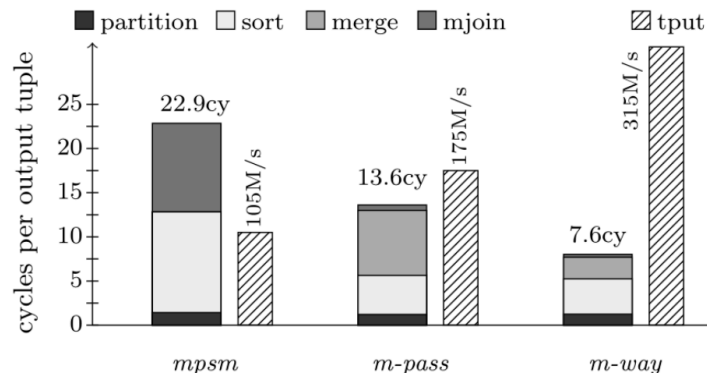
- For smaller relation sizes (e.g., up to 6.4 billion tuples), **mpsm** generally has the fastest execution time.
- For the largest workload tested (**12.8 billion tuples**), the **m-way** algorithm demonstrates the best execution time (around 30.645 seconds), significantly outperforming both m-pass and mpsm (which take over 50 seconds).



Performance Breakdown (Cycles per Output Tuple)

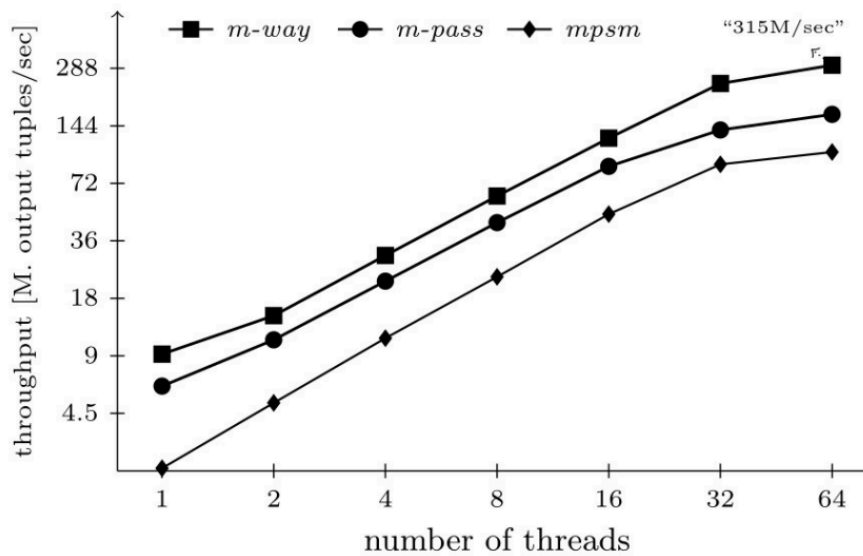
The cost is broken down into four phases: partition, sort, merge, and join (**mjoin**).

- The **m-way** algorithm has the lowest overall cost (7.6 cycles/tuple) and the highest throughput (315 M/s).
- The **sort phase** is the dominant cost component for both mpsm and m-pass, while m-way manages to make the **merge phase** dominant due to its efficiency in sorting.



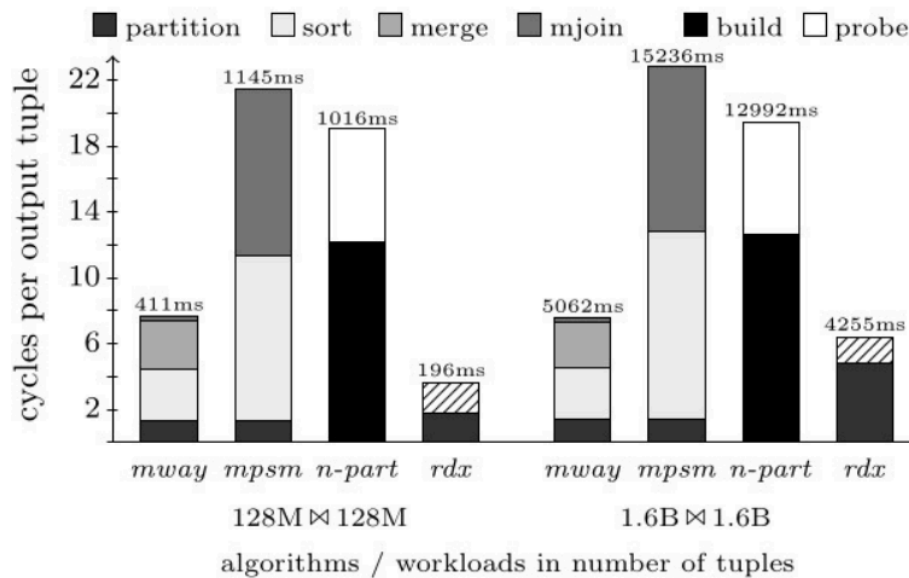
Scalability

- The **m-way** algorithm exhibits the best scalability, maintaining the highest throughput (M. output tuples/sec) as the number of threads increases up to 64.
- The m-way approach achieved a peak throughput of nearly 288 M. output tuples/sec, compared to mpsm's peak of around 100 M. output tuples/sec.



Sort vs. Hash Join Comparison

This final evaluation compares the most optimized SMJ algorithm (**m-way**) against the optimized Hash Join algorithm (**radix**) across various workloads.



Performance Breakdown (Cycles per Output Tuple)

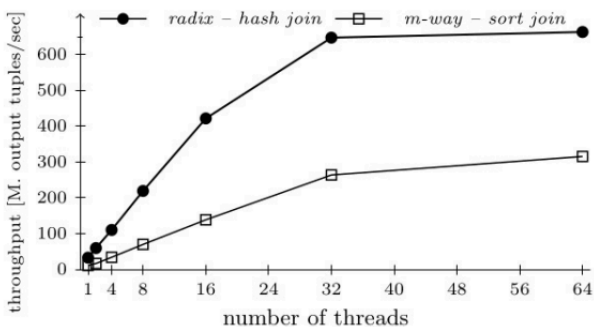
Workload (Tuples)	Best Algorithm	Cycles/Tuple	Dominant Cost in m-way	Dominant Cost in rdx
128M×128M	Radix Hash Join (rdx)	3.58ms	Merge	Build/Probe
1.6B×1.6B	m-way\$	4.255ms	Sort	Build/Probe
1.6B×6.4B	Radix Hash Join (rdx)	3.972ms	Merge	Build/Probe

- **Radix Hash Join** is generally faster for smaller, balanced workloads (128M×128M) and for unbalanced, larger workloads (1.6B×6.4B).
- **m-way\$** is faster for large, balanced workloads (1.6B×1.6B).
- The dominant cost for hash joins (rdx) remains the **Build/Probe** phase, while the dominant cost for m-way is split between **Sort** and **Merge** depending on the specific workload.

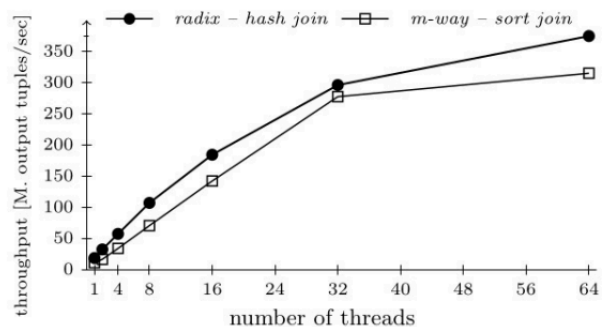
Scalability (Throughput vs. Cores)

The results show different scaling behaviors based on the relation size:

1. **Small Workload (128M tuples):** The **Radix Hash Join** scales far better than the m-way sort join, achieving a much higher maximum throughput.
2. **Large Workload (1.6B tuples):** Both algorithms scale well, but the **Radix Hash Join** slightly edges out the m-way sort join in peak throughput.



(a) 977 MiB × 977 MiB (128 million 8-byte tuples)



(b) 11.92 GiB × 11.92 GiB (1.6 billion 8-byte tuples)

Conclusion

The experimental data confirms that **no single join algorithm is universally superior**. Both **Hash Join** (optimized via Radix partitioning) and **Sort-Merge Join** (optimized via SIMD and multi-way merging) are prevailing classes of join algorithms, and the optimal choice depends on the workload size and data characteristics.

References

- Albutiu, et al., Massively Parallel Sort-Merge Joins in Main Memory Multi-Core Database Systems, VLDB, 2012.
- Balkesen, et al., Main-Memory Hash Joins on Multi-Core CPUs: Tuning to the Underlying Hardware, ICDE, 2013.
- Balkesen, et al., Multi-Core, Main-Memory Joins: Sort vs. Hash Revisited, VLDB, 2013.
- Blanas, et al., Design and Evaluation of Main Memory Hash Join Algorithms for Multi-core CPUs, SIGMOD, 2011.
- Boncz, Manegold, Kersten, Database Architecture Optimized for the new Bottleneck: Memory Access, VLDB1999.
- DeWitt, Gerber, Multiprocessor Hash-Based Join Algorithms, VLDB, 1985.
- Inoue, et al., AA-Sort_ A New Parallel Sorting Algorithm for Multi-Core SIMD Processors, PACT2007.
- Shatdal, Kant, Naughton, Cache conscious algorithms for relational query processing, VLDB1994.